

HPC Distributed Resource Manager

R-322 — Operating Systems I

Juan Ignacio Gorgo
Camila Sarasúa
Lucio Oviedo Dell'Orefice
Santiago Salvático

June 23, 2026

Contents

1	Project Overview	2
2	Erlang Component Architecture	2
2.1	Module Overview	2
2.2	Process Architecture	2
2.3	Test Environment Convention	3
2.4	Startup Sequence	3
3	Communication Protocol	3
3.1	Inter-Agent Protocol (C to C)	3
3.2	Local Interface (Erlang to C Agent)	4
3.3	Sequence Diagram: Successful Job Lifecycle	4
3.4	Successful Job Lifecycle	4
4	Deadlock Prevention Strategy	5
4.1	Problem Description	5
4.2	Strategy: Global Resource Ordering	5
4.3	Why This Works: Step-by-Step Example	5
5	Event Logger	6
5.1	Design	6
5.2	Log Record	6
5.3	Example Output	6
5.4	Design Rationale	6
6	Team Contributions	6
7	Problems Encountered and Solutions	7
7.1	Job ID Collision Between Nodes on the Same IP	7
7.2	Early Closing Socket	7
7.3	Logger Process Id Registration	7
7.4	TCP Read Blocking vs. Erlang Message Passing	7

1 Project Overview

This report describes the design and implementation of a distributed resource manager for a simulated HPC cluster. Each node consists of two cooperating components: a **Resource Manager Agent** written in C, and a **Job Scheduler** written in Erlang. The components communicate via TCP on localhost, while C agents communicate with each other over the network using a common ASCII protocol.

The Erlang side is responsible for job generation, resource request orchestration, deadlock prevention, and event logging. The C side manages local resources, handles concurrent connections via `epoll`, and implements UDP-based node discovery.

2 Erlang Component Architecture

2.1 Module Overview

The Erlang component is structured into four modules with clearly separated responsibilities:

Module	Responsibility
<code>header.hrl</code>	Shared macros, records, and resource ordering constants
<code>erlang_c_bridge</code>	TCP connection management with the local C agent
<code>job_scheduler</code>	Job generation, scheduling logic, and state management
<code>event_logger</code>	Asynchronous event logging to file

2.2 Process Architecture

The system consists of two main components per node: a C agent and an Erlang scheduler. The Erlang side uses the actor model — each logical unit runs as an independent process communicating exclusively through message passing, with no shared memory. The C agent handles all network I/O using `epoll`, delegating resource logic to internal modules connected through a well-defined interface.

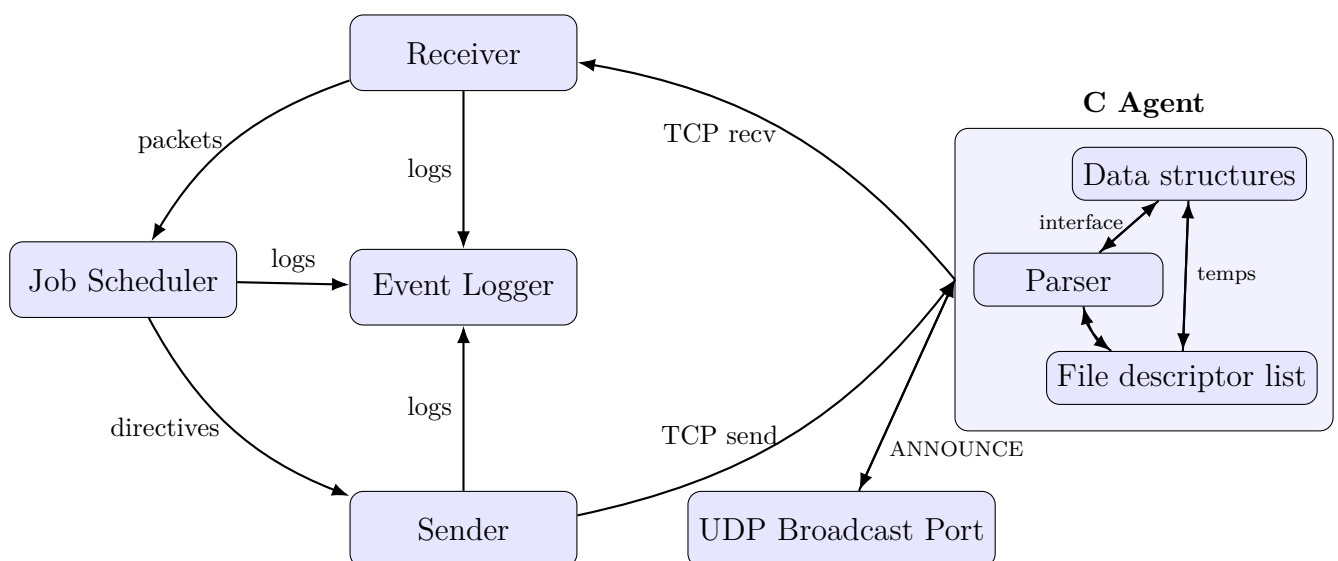


Figure 1: System communication architecture: Erlang processes and C agent internals

2.3 Test Environment Convention

To simplify deadlock testing without the constraints of global resource ordering, a `ENV` environment variable convention was introduced. When `ENV` is set to `"test"`, the `generate_job_request` function skips the resource sorting step, allowing jobs to request resources in arbitrary order. This makes it possible to deliberately trigger the deadlock scenario described in Section 4 and verify that the system detects or recovers from it.

In production (`ENV` unset or any other value), global ordering is always applied and deadlock is structurally prevented.

2.4 Startup Sequence

2.5 C Agent Startup Sequence

The C agent is launched with five arguments: IP address, TCP port, and the available CPU, RAM, and GPU resources. The startup sequence is as follows:

1. The `ServerContext` is initialized with the provided arguments and a new `epoll` instance is created.
2. Three sockets are opened: a public TCP listener bound to the node's LAN IP, a local TCP listener bound to `127.0.0.1` exclusively for the Erlang scheduler, and a UDP socket for broadcast discovery.
3. Three `timerfd` instances are configured: a one-shot TCP timer (2 seconds), a periodic UDP announce timer (every 1 second), and a periodic garbage collector timer (every 5 seconds).
4. The three timers and the UDP socket are registered in `epoll`. The TCP sockets are intentionally **not** registered yet.
5. A thread pool of 4 worker threads is initialized and begins waiting for tasks.
6. The main `epoll` loop starts. During the first 2 seconds, only UDP traffic is processed — the node broadcasts its presence and collects announcements from peers already on the network.
7. When the TCP timer expires, both TCP sockets are added to `epoll` and the agent begins accepting connections from other agents and from the local Erlang scheduler.

When `erlang_c_bridge:init/0` is called:

1. Spawns the `event_logger` process and registers it under the atom `eventLoggerId`.
2. Spawns the `job_scheduler` process.
3. Calls `connect/3`, which attempts up to `?TRIES` TCP connections to the C agent.
4. On success, spawns `conn_handler`, which spawns `sender` and `receiver` and notifies the scheduler with their PIDs.
5. On exhaustion of retries, logs the failure and calls `erlang:halt/0`.

3 Communication Protocol

3.1 Inter-Agent Protocol (C to C)

C agents communicate directly with each other over TCP using a line-oriented ASCII protocol, where each message is terminated by `\n`. This protocol is used to reserve and release resources on remote nodes.

Messages sent between agents:

```
1 RESERVE <job_id> <resource_name> <amount>
2 GRANTED <job_id>
3 DENIED <job_id>
4 RELEASE <job_id> <resource_name> <amount>
```

Where `job_id` is a unique integer generated by the Erlang scheduler, `resource_name` is one of `cpu`, `mem`, or `gpu`, and `amount` is the requested quantity.

Node discovery is handled via UDP broadcast. Each agent periodically announces its presence with:

```
1 ANNOUNCE <port> <cpu:N> <mem:N> <gpu:N>
```

The sender's IP is extracted directly from `recvfrom()` rather than included in the message. If no announcement is received from a node within 15 seconds, it is considered dead and removed from the known nodes table.

3.2 Local Interface (Erlang to C Agent)

The `sender` process forwards directives from the scheduler to the C agent:

```
1 GET_NODES
2 JOB_REQUEST <job_id> @<ip>:<port>:<resource>:<amount> ...
3 JOB_RELEASE <job_id>
```

The `receiver` process listens on the same socket and forwards responses to the scheduler:

```
1 NODES <ip>:<port>:<res>:<amount>:...;<ip>:...
2 JOB_GRANTED <job_id>
3 JOB_DENIED <job_id>
4 JOB_TIMEOUT <job_id>
```

3.3 Sequence Diagram: Successful Job Lifecycle

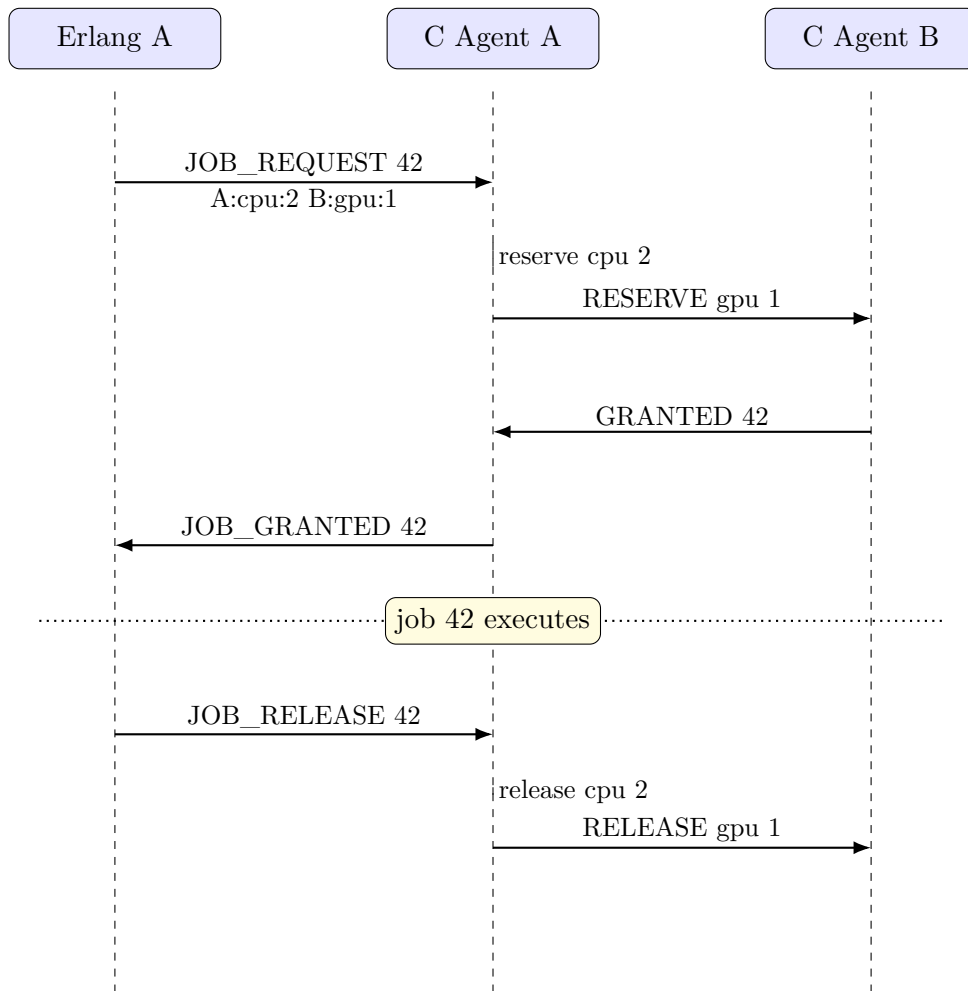


Figure 2: Successful job execution and release.

4 Deadlock Prevention Strategy

4.1 Problem Description

Distributed deadlock occurs when two jobs each hold a resource the other needs, forming a circular wait. The classic scenario from the assignment:

- **Job1** (from Node A): needs 2 CPUs from Node A and 1 GPU from Node B.
- **Job2** (from Node B): needs 1 GPU from Node B and 2 CPUs from Node A.

If both jobs acquire their first resource simultaneously, each blocks waiting for the resource held by the other - a deadlock.

4.2 Strategy: Global Resource Ordering

The chosen strategy eliminates the **circular wait** condition (Coffman condition 4) by enforcing a global ordering on all resource requests before sending them.

Before sending a `JOB_REQUEST`, the scheduler sorts the list of required resources by the pair `(NodeIp, ResourceType)` in ascending order. The type order is defined centrally in `header.hrl`:

```
1 -define(RES_ORDER(Type),
2     case Type of
3         cpu -> 1;
4         gpu -> 2;
5         ram -> 3
6     end
7 ).
```

The sort is applied in `generate_job_request/2`:

```
1 SortedResources = lists:sort(
2     fun ({Ip1, Resource1, _}, {Ip2, Resource2, _}) ->
3         {Ip1, Resource1} =< {Ip2, Resource2}
4     end,
5     AvailableResources
6 ),
```

4.3 Why This Works: Step-by-Step Example

With the ordering invariant enforced, the deadlock scenario resolves as follows:

1. Both Job1 and Job2 sort their resource lists. Since "A" < "B" lexicographically, both lists start with resources from Node A.
2. Both jobs compete for Node A's CPUs first. One wins; the other waits in Node A's FIFO queue.
3. The winning job (Job1) then requests Node B's GPU and acquires it.
4. Job1 finishes, releases both resources. Job2 then acquires Node A's CPUs, then Node B's GPU, and completes normally.

The key invariant: *no job can hold a resource from a "later" node while waiting for a resource from an "earlier" node*. Circular wait is structurally impossible.

5 Event Logger

5.1 Design

The logger runs as a dedicated Erlang process, registered under the atom `eventLoggerId`. Any module logs an event by calling:

```
1 event_logger:log_event(Status, SrcMethod, Detail, JobInvolved)
```

This builds a `#logInfo{}` record and sends it as a message to the logger, which writes it to `event_logger.txt` in append mode using raw file I/O.

5.2 Log Record

```
1 -record(logInfo, {
2     status,           % ok / error / close
3     src_method,       % {Module, Function}
4     detail,           % atom / string describing the event
5     job_involved,     % job_id integer / none
6     timestamp         % {Hour, Min, Sec}
7 }).
```

5.3 Example Output

```
1 ok {erlang_c_bridge,init} initialization none {14,32,10}
2 ok {erlang_c_bridge,connect} "CONNECTION SUCCESSFUL" none {14,32,10}
3 ok {job_scheduler,init} "[job_scheduler] sender_pid received" none
  {14,32,11}
4 ok {job_scheduler,init} "[job_scheduler] job_request" 5 {14,32,16}
5 ok {job_scheduler,init} "[job_scheduler] job_granted" 5 {14,32,16}
```

5.4 Design Rationale

Registering the logger under a fixed atom avoids threading the logger PID through every function call. Any process in the system can log at any time without holding a reference.

6 Team Contributions

Member	Contributions
[Lucio Oviedo Dell'Orefice]	C Agent: epoll server, TCP handling, non-blocking I/O.
[Juan Ignacio Gorgo]	C Agent: resource management, FIFO queues, GRANTED/DENIED logic.
[Camila Sarasúa]	Erlang: job scheduler, deadlock prevention, job state machine.
[Santiago Salvático]	Erlang: <code>erlang_c_bridge</code> , <code>event_logger</code> , <code>header.hrl</code> , documentation, integration.

7 Problems Encountered and Solutions

7.1 Job ID Collision Between Nodes on the Same IP

Problem: The assignment specifies that two nodes may share the same IP address as long as they use different ports. Since each node's Erlang scheduler generates job IDs independently using `erlang:unique_integer/1`, two schedulers on the same IP can produce identical job IDs. When a third agent receives a `RESERVE 42 cpu 2` message, it cannot determine which node sent it based on the job ID alone.

Solution: The C agent already has access to the file descriptor (`fd`) of the incoming TCP connection via the `SOCKET` parameter in `resource_adapter_patch`. Since the kernel guarantees that no two active connections share the same `fd` within a process, the pair (`job_id`, `fd`) is globally unique:

- `fd` uniquely identifies which node sent the message.
- Within the same connection (`fd`), `erlang:unique_integer` guarantees the job ID never repeats.

The solution required no changes to the network protocol — the `fd` is already available throughout the event handling pipeline and only needed to be incorporated as part of the job key in the active jobs table.

7.2 Early Closing Socket

Problem: When the process that opened a TCP socket exits, Erlang closes the socket automatically. This caused the connection to drop when `conn_handler` spawned children and then terminated.

Solution: The socket ID is passed directly to processes `sender` and `receiver` at spawn time. Since these are long-lived processes, the socket remains open for the session's duration.

7.3 Logger Process Id Registration

Problem: Passing the logger PID through every function call created unnecessary coupling between modules.

Solution: The logger is registered under the atom `eventLoggerId` via `register/2`. All modules send log messages to the atom directly.

7.4 TCP Read Blocking vs. Erlang Message Passing

Problem: A process blocked in `gen_tcp:recv` with `{active, false}` cannot receive Erlang messages simultaneously. The alternative, `{active, true}`, delivers TCP data directly to the process mailbox — but if the C agent sends messages faster than the process consumes them, the mailbox can exhaust memory.

Solution: We kept `{active, false}` and separated inbound and outbound traffic into two dedicated processes:

- `receiver`: blocks on `gen_tcp:recv` waiting for C agent responses.
- `sender`: blocks on `receive` waiting for scheduler directives.

Each process has a single blocking responsibility, avoiding both problems.